EdUHK Robotics Team

TECHNICAL / TEAM DOCUMENT

Technical Documentation Template

Robocon 2026 R2 Arduino IMU and encoder receiver example

Author Steven Zhang Yancheng
Date 2026 年 8 月 23 日
Version v1.4

OVERVIEW

Template structure

This template translates the EdUHK Robotics Team's web/CSS visual language into a practical A4 format for technical documentation. It combines the team's green accent system, official horizontal logo, clean sans-serif typography, reusable cards, tables, and a light-blue code area.

Included code example

The example in this document is the **Robocon 2026 EdUHK Robotics Team R2 Arduino IMU and encoder receiver program**. It is reproduced from `enc_imu_may31.ino` and credited to **Steven Zhang Yancheng**.
Source: Steven-ZYC/Robocon2026_r2 on GitHub (dev branch)

Program role

The Arduino sketch reads two quadrature encoders, parses LPBUS IMU packets received on `Serial3`, combines heading, angular rate, acceleration, and encoder counts into one payload, and appends a CRC-8/ATM checksum before serial output.

Area	Implementation
Controller	Arduino Mega 2560 serial and interrupt interfaces
Encoders	Two AMT103 quadrature channels, decoded at four times resolution
IMU	LPBUS packet parser on `Serial3` at 115200 baud
Output	Timestamped IMU/encoder payload with CRC-8/ATM
Credit	Steven Zhang Yancheng, Robocon 2026 EdUHK Robotics Team R2

Reusable content blocks

Design note

Use neutral cards for context, constraints, assumptions, or supplementary explanations. The green-accented information card is better for key decisions, warnings, and important takeaways.

Template usage

Change the metadata commands near the top of the source file, replace the sample body, and keep only the components needed for the final report.

Image placement

Replace the placeholder below with a normal `figure` environment and `\includegraphics` when an image, CAD render, or system diagram is available.

IMAGE

Robot photograph / CAD render / system diagram

Example table

Module	Description	Status
Encoder input	AMT103 X/Y quadrature channels	Active
IMU link	LPBUS packets received over `Serial3`	Active
Packet output	Combined telemetry with CRC-8/ATM	Testing

Replace the sample image and table with project-specific content.

CODE EXAMPLE

Arduino IMU and encoder receiver

Robocon 2026 EdUHK Robotics Team R2

Arduino program for receiving IMU and encoder data.
By Steven Zhang Yancheng
Source: Robocon2026_r2/enc_imu_may31.ino

Listing 1: Robocon 2026 EdUHK Robotics Team R2 Arduino IMU and encoder receiver program - by Steven Zhang Yancheng

```
#include <Arduino.h>
#include <math.h>
#include <stdio.h>
#include <string.h>

/* ==================== Encoder (AMT103) 引腳配置 ==================== */
// Pin 2,3 為 X 軸; Pin 18,19 為 Y 軸
static const uint8_t ENC_X_A = 20,  ENC_X_B = 21;
static const uint8_t ENC_Y_A = 18, ENC_Y_B = 19;

/* X 編碼器變數 */
volatile long enc_x_cnt = 0;
volatile uint8_t enc_x_last = 0;

/* Y 編碼器變數 */
volatile long enc_y_cnt = 0;
volatile uint8_t enc_y_last = 0;

/* 四倍頻解碼表 */
static inline int8_t quad(uint8_t p, uint8_t c) {
  static const int8_t t[16] = {
    0, 1, -1, 0,
    -1, 0, 0, 1,
    1, 0, 0, -1,
    0, -1, 1, 0
  };
  return t[(p << 2) | c];
}

static inline uint8_t rdAB(uint8_t a, uint8_t b) {
  return (digitalRead(a) ? 1 : 0) | (digitalRead(b) ? 2 : 0);
}

/* X 軸中斷回調 (原本的 E1) */
void isr_x() {
  uint8_t ab = rdAB(ENC_X_A, ENC_X_B);
  enc_x_cnt += quad(enc_x_last, ab);
  enc_x_last = ab;
}

```

```
/* Y 軸中斷回調 (原本的 E2) */
void isr_y() {
  uint8_t ab = rdAB(ENC_Y_A, ENC_Y_B);
  enc_y_cnt += quad(enc_y_last, ab);
  enc_y_last = ab;
}

/* ===================== CRC8 ATM ===================== */
static uint8_t crc8(const uint8_t* d, size_t n) {
  uint8_t c = 0;
  for (size_t i = 0; i < n; i++) {
    c ^= d[i];
    for (uint8_t b = 0; b < 8; b++)
      c = (c & 0x80) ? (uint8_t)((c << 1) ^ 0x07) : (uint8_t)(c << 1);
  }
  return c;
}

static void print_crc_line(const char* s) {
  uint8_t c = crc8((const uint8_t*)s, strlen(s));
  Serial.print(s);
  Serial.print(" crc=");
  if (c < 16) Serial.print('0');
  Serial.println(c, HEX);
}

/* ===================== LPBUS IMU Parser ===================== */
enum PS { WAIT, CMD, IDX, LEN, DATA, LRC, CR, LF };
PS ps = WAIT;
uint8_t cmd, idx, len, lrc, sum;
uint8_t buf[32], blen;

static inline int16_t rd16(const uint8_t* p) {
  return (int16_t)(p[0] | (p[1] << 8));
}

static uint32_t pkg_id = 0;
static unsigned long t0 = 0;


/* ===================== 輸出端口选择 ===================== */
// true  = USB Serial 輸出
// false = UART Serial2 輸出, Mega 2560: TX2=D16, RX2=D17
static const bool OUTPUT_TO_USB = true;

static Stream* outPort() {
  return OUTPUT_TO_USB ? (Stream*)&Serial : (Stream*)&Serial2;
}

/* ===================== 輸出數據包 ===================== */
static void emit_package(float hdg, float rate, float ax, float ay, float az) {
  unsigned long t = millis() - t0;

  long cur_x, cur_y;
  noInterrupts();
  cur_x = enc_x_cnt;
  cur_y = enc_y_cnt;
  interrupts();

```

```
  char payload[256];
  char s_hdg[12], s_rate[12], s_ax[12], s_ay[12], s_az[12];

  dtostrf(hdg, 1, 2, s_hdg);
  dtostrf(rate, 1, 2, s_rate);
  dtostrf(ax, 1, 3, s_ax);
  dtostrf(ay, 1, 3, s_ay);
  dtostrf(az, 1, 3, s_az);

  // 輸出, ENC = X, Y
  // CRC 只計算 payload, 不包含 <、*XX、>
  int n = snprintf(payload, sizeof(payload),
    "ID=%lu T=%lu IMU=%s,%s,%s,%s,%s ENC=%ld,%ld",
    (unsigned long)pkg_id, (unsigned long)t,
    s_hdg, s_rate, s_ax, s_ay, s_az,
    cur_x, cur_y);

  if (n > 0 && (size_t)n < sizeof(payload)) {
    uint8_t c = crc8((const uint8_t*)payload, strlen(payload));

    Stream* out = outPort();

    Serial.print('<');
    Serial.print(payload);
    Serial.print(",*");
    if (c < 16) Serial.print('0');
    Serial.print(c, HEX);
    Serial.println('>');

    pkg_id++;
  }
}

/* ==================== IMU 數據處理 ==================== */
void imuPump() {
  while (Serial3.available()) {
    uint8_t b = (uint8_t)Serial3.read();
    switch (ps) {
      case WAIT: if (b == ':') { ps = CMD; sum = 0; blen = 0; } break;
      case CMD:  cmd = b; sum += b; ps = IDX; break;
      case IDX:  idx = b; sum += b; ps = LEN; break;
      case LEN:  len = b; sum += b; blen = 0;
                 if (len == 0 || len > sizeof(buf)) ps = WAIT; else ps = DATA; break;
      case DATA: buf[blen++] = b; sum += b; if (blen >= len) ps = LRC; break;
      case LRC:  lrc = b; ps = CR; break;
      case CR:   ps = (b == 0x0D) ? LF : WAIT; break;
      case LF:
        ps = WAIT;
        if (b == 0x0A && sum == lrc && cmd == 0x0B && len == 10) {
          emit_package(
            rd16(&buf[0]) / 100.0f,
            rd16(&buf[2]) / 50.0f,
            rd16(&buf[4]) / 1000.0f,
            rd16(&buf[6]) / 1000.0f,
            rd16(&buf[8]) / 1000.0f
          );
        }
        break;
    }
```

```
  }
}

/* ==================== Setup & Loop ==================== */
void setup() {
  pinMode(ENC_X_A, INPUT); pinMode(ENC_X_B, INPUT);
  pinMode(ENC_Y_A, INPUT); pinMode(ENC_Y_B, INPUT);

  enc_x_last = rdAB(ENC_X_A, ENC_X_B);
  enc_y_last = rdAB(ENC_Y_A, ENC_Y_B);

  attachInterrupt(digitalPinToInterrupt(ENC_X_A), isr_x, CHANGE);
  attachInterrupt(digitalPinToInterrupt(ENC_X_B), isr_x, CHANGE);
  attachInterrupt(digitalPinToInterrupt(ENC_Y_A), isr_y, CHANGE);
  attachInterrupt(digitalPinToInterrupt(ENC_Y_B), isr_y, CHANGE);

  Serial.begin(115200);    // USB Serial
  Serial2.begin(115200);   // UART output: TX2=D16, RX2=D17
  Serial3.begin(115200);   // IMU: TX3=D14, RX3=D15
  t0 = millis();
}

void loop() {
  imuPump();
}
```